Programación Concurrente y Paralela

Departamento de Computación – FCEyN – UBA

Segundo cuatrimestre de 2026

Simulacro 2do. parcial – 2do. cuat. de 2026

Nombre y apellido: _____

Nº orden: _____ L.U.: _____ Cant. hojas: ____

1	2	3	4	Nota

- Cada ejercicio debe realizarse en hojas separadas y numeradas. Debe identificarse cada hoja con nombre, apellido, LU y número de orden. Hojas sin identificar no serán corregidas.
- Entregue esta hoja junto al examen, la misma **no** se incluye en la cantidad total de hojas entregadas.
- **Cada código o pseudocódigo debe estar bien explicado y justificado. ¡Obligatorio!**
- Toda suposición o decisión que tome deberá justificarla adecuadamente. Si la misma no es correcta o no se condice con el enunciado no será tomada como válida y será corregida acorde.
- La devolución de los exámenes corregidos es personal. Los pedidos de revisión se realizarán por escrito, antes de retirar el examen corregido del aula.
- Los parciales tienen dos notas: I (Insuficiente) y A (Aprobado).
- Para tener la nota aprobada es necesario tener al menos dos ejercicios calificados como bien y el restante al menos regular. Además, alguno de los ejercicios calificados como bien debe ser el de STM o el de mensajes.

Ejercicio 1: Centro de atención. Una empresa de internet tiene una línea de atención al cliente que colapsa en cada corte masivo, cuando a las consultas de siempre se suman todos los que se quedaron sin servicio. Para el próximo corte, decidió atender con agentes de IA, y antes de ponerlos a funcionar quiere simular la línea con STM.

En la simulación, las llamadas son de dos tipos, urgentes (clientes sin servicio) y comunes (consultas de facturación). Cada tipo tiene su propia cola de espera, con lugar para K llamadas. Cada cliente que llama es un thread que ejecuta `llamar` y deja su llamada en la cola del tipo que corresponde. Si la cola está llena, el cliente espera a que se libere un lugar. Una vez en la cola, espera en la línea hasta que un agente toma su llamada, y recién ahí `llamar` retorna.

Cada agente es un thread que ejecuta `agente`. Atiende de a una llamada por vez y siempre prefiere una urgente, si hay. Para que las comunes no queden relegadas, un agente que atendió 3 urgentes seguidas tiene que atender una común antes de volver a una urgente, si hay alguna común esperando.

Se cuenta con lo siguiente:

```
data Tipo = Urgente | Comun
type Llamada = Int
atender :: Llamada -> IO ()   -- El agente atiende la llamada
```

Se cuenta además con el siguiente esqueleto:

```
data Centro = ...

nuevoCentro :: Int -> IO Centro
nuevoCentro k = ...

llamar :: Centro -> Tipo -> Llamada -> IO ()
llamar centro tipo llamada = ...

agente :: Centro -> IO ()
agente centro = ...
```

a) Complete el esqueleto usando STM, incluida la representación del centro. La solución no debe tener deadlocks ni race conditions. Justifique todas las decisiones tomadas.

b) Indique si su solución puede sufrir starvation, justificando a partir de cómo implementa Haskell las transacciones. Si puede, describa una traza que lleve a esa situación.

c) La empresa contrata un centro tercerizado como respaldo, con sus propias colas y agentes. Todos los clientes aceptan que los atienda el primero de los dos centros que tome su llamada. Explique, sin escribir código, cómo adaptaría su solución para estos clientes. Justifique.

Solución.

```
import Control.Concurrent.STM
import Control.Monad

type EnLinea = (Llamada, TVar Bool)

data Centro = Centro Int (TVar [EnLinea]) (TVar [EnLinea])

nuevoCentro :: Int -> IO Centro
nuevoCentro k = do
  urgentes <- newTVarIO []
  comunes  <- newTVarIO []
  return (Centro k urgentes comunes)

encolar :: Int -> TVar [EnLinea] -> EnLinea -> STM ()
encolar capacidad cola enLinea = do
  esperando <- readTVar cola
  when (length esperando >= capacidad) retry
  writeTVar cola (esperando ++ [enLinea])

esperarAtencion :: TVar Bool -> STM ()
esperarAtencion atendida = do
  listo <- readTVar atendida
  when (not listo) retry

llamar :: Centro -> Tipo -> Llamada -> IO ()
llamar (Centro k urgentes comunes) tipo llamada = do
  atendida <- newTVarIO False
  case tipo of
    Urgente -> atomically (encolar k urgentes (llamada, atendida))
    Comun   -> atomically (encolar k comunes (llamada, atendida))
  atomically (esperarAtencion atendida)

tomarDe :: TVar [EnLinea] -> STM Llamada
tomarDe cola = do
  esperando <- readTVar cola
  case esperando of
    [] -> retry
    ((llamada, atendida) : resto) -> do
      writeTVar cola resto
      writeTVar atendida True
      return llamada

tomarUrgente :: Centro -> TVar Int -> STM Llamada
tomarUrgente (Centro _ urgentes _) seguidas = do
  llamada <- tomarDe urgentes
  n <- readTVar seguidas
  writeTVar seguidas (n + 1)
  return llamada

tomarComun :: Centro -> TVar Int -> STM Llamada
tomarComun (Centro _ _ comunes) seguidas = do
  llamada <- tomarDe comunes
  writeTVar seguidas 0
  return llamada

elegir :: Centro -> TVar Int -> STM Llamada
elegir centro seguidas = do
  n <- readTVar seguidas
  if n < 3
    then orElse (tomarUrgente centro seguidas) (tomarComun centro seguidas)
    else orElse (tomarComun centro seguidas) (tomarUrgente centro seguidas)

agente :: Centro -> IO ()
agente centro = do
```

```
65    seguidas <- newTVarIO 0
66    forever (do
67      llamada <- atomically (elegir centro seguidas)
68      atender llamada)
```

Ejercicio 2: Trapitos. Si uno va a un partido de fútbol en auto, no puede escaparle a una constante en la vida del hincha: los trapitos. Pero estamos en el siglo XXI, la barra se modernizó y quiere un simulador para probar qué tan efectivos son distintos modelos de organización. A cambio de un palco vitalicio, la barra nos encargó que simulemos el siguiente sistema de trapitos y lugares.

Los lugares son finitos y están contados. La barra controla N lugares de estacionamiento que se alquilan a los autos que van llegando. ¿Cómo se hace eso?

Cada hincha motorizado que llega pide un lugar y espera. Un trapito al azar (hay una cantidad arbitraria) toma el pedido y busca un lugar libre donde estacionar. Si no hay, espera; si hay, reserva el lugar, le avisa a la barra que ese lugar fue reservado y le avisa al hincha qué lugar le fue asignado. La barra, que es única, registra la reserva y cada 5 minutos suma 1 al costo del alquiler del lugar. Arranca en 1.

Al terminar el partido (o al querer irse, si se va antes de que termine), el hincha le pregunta a la barra cuánto tiene que pagar e intenta pagar ese monto. Si en el medio pasaron otros 5 minutos y el precio aumentó, tiene que volver a consultar y reintentar, porque no puede retirar el auto mientras pague menos de lo debido. Cuando el pago alcanza, la barra lo deja retirarse y libera el lugar para otro hincha.

Implementar utilizando comunicación por medio de mensajes, con las reglas vistas en clase[1]:

a) Los threads `trapito`, `hincha` y `barra`. Habrá T trapitos, H hinchas y 1 barra. Pueden usar funciones y threads auxiliares. Para simular al hincha viendo el partido, llamen a una función imaginaria `verPartido()` después de que el hincha haya estacionado y antes de retirar el auto. La solución debe ser libre de deadlocks y se tiene que justificar. Se valora el uso de diagramas para dicha explicación.

b) *(Esta parte NO entra en el parcial, pero la pusimos porque nos gusta la discusión.)* ¿Qué pasaría si, en vez de una barra, hubiera muchas? ¿Qué cambios harían?

Solución. Si uno va a un partido de fútbol en auto, no puede escaparle a una constante en la vida del hincha: los trapitos. Pero estamos en el siglo XXI, la barra se modernizó y quiere un simulador para probar qué tan efectivos son distintos modelos de organización. A cambio de un palco vitalicio, la barra nos encargó que simulemos el siguiente sistema de trapitos y lugares.

Los lugares son finitos y están contados. La barra controla N lugares de estacionamiento que se alquilan a los autos que van llegando. ¿Cómo se hace eso?

Cada hincha motorizado que llega pide un lugar y espera. Un trapito al azar (hay una cantidad arbitraria) toma el pedido y busca un lugar libre donde estacionar. Si no hay, espera; si hay, reserva el lugar, le avisa a la barra que ese lugar fue reservado y le avisa al hincha qué lugar le fue asignado. La barra, que es única, registra la reserva y cada 5 minutos suma 1 al costo del alquiler del lugar. Arranca en 1.

Al terminar el partido (o al querer irse, si se va antes de que termine), el hincha le pregunta a la barra cuánto tiene que pagar e intenta pagar ese monto. Si en el medio pasaron otros 5 minutos y el precio aumentó, tiene que volver a consultar y reintentar, porque no puede retirar el auto mientras pague menos de lo debido. Cuando el pago alcanza, la barra lo deja retirarse y libera el lugar para otro hincha.

Implementar utilizando comunicación por medio de mensajes, con las reglas vistas en clase[2]:

a) Los threads `trapito`, `hincha` y `barra`. Habrá T trapitos, H hinchas y 1 barra. Pueden usar funciones y threads auxiliares. Para simular al hincha viendo el partido, llamen a una función imaginaria `verPartido()` después de que el hincha haya estacionado y antes de retirar el auto. La solución debe ser libre de deadlocks y se tiene que justificar. Se valora el uso de diagramas para dicha explicación.

b) *(Esta parte NO entra en el parcial, pero la pusimos porque nos gusta la discusión.)* ¿Qué pasaría si, en vez de una barra, hubiera muchas? ¿Qué cambios harían?

Solución:

```
1  package main
2
3  import (
```

[1] Sin memoria compartida y con el `nil` propio de cada tipo. Los canales se comportan igual que en Go.

```
    "strconv"
    "sync"
    "time"
)

func lugares(cant int, disponibles chan<- int, liberados <-chan int) {
    for i := 0; i < cant; i++ {
        disponibles <- i
    }
    for lugar := range liberados {
        disponibles <- lugar
    }
}

type PedidoEstacionamiento struct {
    cliente              string
    lugarEstacionamiento chan int
}

type Estacionamiento struct {
    cliente              string
    lugarEstacionamiento int
}

type ConsultaMonto struct {
    cliente string
    monto   chan int
}

type Pago struct {
    cliente string
    monto   int
    ok      chan bool // true: pago aceptado, false: monto insuficiente
}

func trapito(estacionar <-chan PedidoEstacionamiento, disponibles <-chan int, barra chan<-
    Estacionamiento) {
    for pedido := range estacionar {
        lugar := <-disponibles
        barra <- Estacionamiento{pedido.cliente, lugar}
        pedido.lugarEstacionamiento <- lugar
        println("Trapito: Hincha", pedido.cliente, "estacionado en lugar", lugar)
    }
}

func barra(trapitos <-chan Estacionamiento, consultas <-chan ConsultaMonto, pagos <-chan Pago,
    liberados chan<- int) {
    // Diccionario cliente -> {lugar, monto}
    montos := make(map[string]int)
    lugares := make(map[string]int)
    recaudacionTotal := 0
    // Ticker para sumar 1 al monto de cada cliente cada 5 minutos
    ticker := time.NewTicker(5 * time.Minute)
    defer ticker.Stop()

    for {
        select {
        case trapito := <-trapitos:
            montos[trapito.cliente] += 1
            lugares[trapito.cliente] = trapito.lugarEstacionamiento
        case consulta := <-consultas:
            consulta.monto <- montos[consulta.cliente]
        case pago := <-pagos:
            if pago.monto < montos[pago.cliente] {
                pago.ok <- false
                continue
```

```
            }
            recaudacionTotal += pago.monto
            println("Barra: Hincha", pago.cliente, "pagó", pago.monto, "y se retiró del lugar",
    lugares[pago.cliente])
            liberados <- lugares[pago.cliente]
            delete(montos, pago.cliente)
            delete(lugares, pago.cliente)
            pago.ok <- true
        case <-ticker.C:
            for cliente := range montos {
                montos[cliente] += 1
            }
        }
    }
}

// verPartido es la función imaginaria que simula al hincha viendo el partido
func verPartido() {}

func hincha(nombre string, estacionar chan<- PedidoEstacionamiento, consultas chan<-
    ConsultaMonto, pagos chan<- Pago) {
    lugarEstacionamiento := make(chan int)
    estacionar <- PedidoEstacionamiento{nombre, lugarEstacionamiento}
    lugar := <-lugarEstacionamiento
    println("Hincha", nombre, "estacionado en lugar", lugar)
    verPartido()
    // Consultar monto y pagar hasta que la barra acepte
    montoChan := make(chan int)
    ok := make(chan bool)
    var monto int
    for {
        consultas <- ConsultaMonto{nombre, montoChan}
        monto = <-montoChan
        pagos <- Pago{nombre, monto, ok}
        if <-ok {
            break
        }
    }
    println("Hincha", nombre, "pagó", monto, "y se retiró del lugar", lugar)
}

func main() {
    cantTrapitos := 3
    cantLugares := 5
    cantClientes := 10
    disponibles := make(chan int, cantLugares)
    estacionar := make(chan PedidoEstacionamiento)
    avisarBarra := make(chan Estacionamiento)
    liberados := make(chan int)
    consultas := make(chan ConsultaMonto)
    pagos := make(chan Pago)
    go lugares(cantLugares, disponibles, liberados)
    go barra(avisarBarra, consultas, pagos, liberados)
    for i := 0; i < cantTrapitos; i++ {
        go trapito(estacionar, disponibles, avisarBarra)
    }
    var wg sync.WaitGroup
    for i := 0; i < cantClientes; i++ {
        wg.Go(func() {
            hincha("Hincha"+strconv.Itoa(i), estacionar, consultas, pagos)
        })
    }
    wg.Wait()
}
```

Ejercicio 3. Un servidor atiende a dos clientes. El cliente i manda sus pedidos por el canal `pedido`$_i$ y recibe las respuestas por `resp`$_i$, todos de tipo `chan bool` y sin buffer. Se consideran dos versiones del servidor, A y B:

```
for {  // cliente 1
  pedido1 <- true
  <-resp1
  usar()
}
```

```
for {  // cliente 2
  pedido2 <- true
  <-resp2
  usar()
}
```

```
// servidor, versión A
for {
  select {
  case <-pedido1: resp1 <- true
  case <-pedido2: resp2 <- true
  }
}
```

```
// servidor, versión B
for {
  <-pedido1; resp1 <- true
  <-pedido2; resp2 <- true
}
```

a) Modele las dos versiones del sistema como composición de LTS. Indique en qué estados valen las proposiciones $pide_i$, $espera_i$, usa_i y $atiende_i$, que se usan en los ítems siguientes.

b) Escriba en LTL las siguientes propiedades:

1. Todo cliente que pide termina usando una respuesta.
2. Un cliente que está esperando su respuesta no vuelve a pedir sin haberla usado antes.
3. Entre dos atenciones al cliente 1, el servidor atiende al cliente 2.

c) Determine si cada versión satisface cada una de las siguientes fórmulas. Para las que no, dé una traza que la viole y decida si pasa a valer bajo fair choice.
1. $\Box\Diamond usa_2$ 2. $\Box(pide_1 \to \Diamond espera_1)$ 3. $\Box\neg(usa_1 \wedge usa_2)$ 4. $\Box(espera_1 \to \bigcirc(espera_1 \vee usa_1))$

Solución.

a) **Los componentes.** Cada cliente tiene tres estados: P (por pedir), E (espera la respuesta) y U (usa). Sus acciones son $P \xrightarrow{\text{pedido}_i} E \xrightarrow{\text{resp}_i} U \xrightarrow{\text{usar}_i} P$.

El servidor A tiene tres estados: 0 (en el `select`), 1 y 2 (recibió el pedido del cliente 1 o 2 y le debe la respuesta). Sus transiciones son $0 \xrightarrow{\text{pedido}_1} 1 \xrightarrow{\text{resp}_1} 0$ y $0 \xrightarrow{\text{pedido}_2} 2 \xrightarrow{\text{resp}_2} 0$.

El servidor B tiene cuatro estados en ciclo: $0 \xrightarrow{\text{pedido}_1} 1 \xrightarrow{\text{resp}_1} 2 \xrightarrow{\text{pedido}_2} 3 \xrightarrow{\text{resp}_2} 0$.

La composición. Los canales no tienen buffer, con lo cual pedido_i y resp_i sincronizan al servidor con el cliente i. usar_i es local. Cada estado es la terna (servidor, cliente 1, cliente 2), y se parte de $(0, P, P)$.

Las proposiciones. $pide_i$, $espera_i$ y usa_i valen cuando el cliente i está en P, E y U. En la versión A, $atiende_i$ vale cuando el servidor está en i. En la versión B, $atiende_1$ vale en el estado 1 y $atiende_2$ en el estado 3.

Versión A (8 estados)

Estado	Transiciones
$(0,P,P)$	$\text{pedido}_1 \to (1,E,P)$, $\text{pedido}_2 \to (2,P,E)$
$(1,E,P)$	$\text{resp}_1 \to (0,U,P)$
$(2,P,E)$	$\text{resp}_2 \to (0,P,U)$
$(0,U,P)$	$\text{usar}_1 \to (0,P,P)$, $\text{pedido}_2 \to (2,U,E)$
$(0,P,U)$	$\text{pedido}_1 \to (1,E,U)$, $\text{usar}_2 \to (0,P,P)$
$(2,U,E)$	$\text{usar}_1 \to (2,P,E)$, $\text{resp}_2 \to (0,U,U)$
$(1,E,U)$	$\text{resp}_1 \to (0,U,U)$, $\text{usar}_2 \to (1,E,P)$
$(0,U,U)$	$\text{usar}_1 \to (0,P,U)$, $\text{usar}_2 \to (0,U,P)$

Versión B (12 estados)

Estado	Transiciones
$(0,P,P)$	$\text{pedido}_1 \to (1,E,P)$
$(1,E,P)$	$\text{resp}_1 \to (2,U,P)$
$(2,U,P)$	$\text{usar}_1 \to (2,P,P)$, $\text{pedido}_2 \to (3,U,E)$
$(2,P,P)$	$\text{pedido}_2 \to (3,P,E)$
$(3,U,E)$	$\text{usar}_1 \to (3,P,E)$, $\text{resp}_2 \to (0,U,U)$
$(3,P,E)$	$\text{resp}_2 \to (0,P,U)$
$(0,U,U)$	$\text{usar}_1 \to (0,P,U)$, $\text{usar}_2 \to (0,U,P)$
$(0,P,U)$	$\text{pedido}_1 \to (1,E,U)$, $\text{usar}_2 \to (0,P,P)$
$(0,U,P)$	$\text{usar}_1 \to (0,P,P)$
$(1,E,U)$	$\text{resp}_1 \to (2,U,U)$, $\text{usar}_2 \to (1,E,P)$
$(2,U,U)$	$\text{usar}_1 \to (2,P,U)$, $\text{usar}_2 \to (2,U,P)$

Ninguna de las dos versiones tiene estados sin salida.

b) 1. $\Box(\mathit{espera}_1 \to \Diamond \mathit{usa}_1) \land \Box(\mathit{espera}_2 \to \Diamond \mathit{usa}_2)$. También se acepta con pide_i en lugar de espera_i, si se interpreta "pide" como "quiere pedir".

2. $\Box(\mathit{espera}_1 \to (\neg \mathit{pide}_1 \,\mathcal{W}\, \mathit{usa}_1)) \land \Box(\mathit{espera}_2 \to (\neg \mathit{pide}_2 \,\mathcal{W}\, \mathit{usa}_2))$. Se usa $\mathcal{W}$ porque la propiedad no exige que la respuesta llegue, solo que no se vuelva a pedir antes. Si escriben $\mathcal{U}$, la fórmula exige además que la use, que es más fuerte de lo pedido.

3. La atención al cliente 1 puede durar varios estados (mientras tanto se mueve el cliente 2), así que hay que separar tres fases: la atención actual, el tramo sin atender al 1 y la atención al 2.

$$\Box\big(\mathit{atiende}_1 \to (\mathit{atiende}_1 \,\mathcal{W}\, (\neg \mathit{atiende}_1 \,\mathcal{W}\, \mathit{atiende}_2))\big)$$

También es correcta $\Box\big((\mathit{atiende}_1 \land \bigcirc \neg \mathit{atiende}_1) \to \bigcirc(\neg \mathit{atiende}_1 \,\mathcal{W}\, \mathit{atiende}_2)\big)$. El error típico es escribir $\Box(\mathit{atiende}_1 \to (\neg \mathit{atiende}_1 \,\mathcal{W}\, \mathit{atiende}_2))$, que es falsa en cuanto $\mathit{atiende}_1$ vale, porque en ese mismo estado ya no vale $\neg \mathit{atiende}_1$. En la versión B la propiedad vale, y en la A no.

c) 1. $\Box\Diamond \mathit{usa}_2$.

Versión A: no vale. El `select` puede elegir siempre al cliente 1: $(0, P, P) \xrightarrow{\text{pedido}_1} (1, E, P) \xrightarrow{\text{resp}_1} (0, U, P) \xrightarrow{\text{usar}_1} (0, P, P) \cdots$ y así para siempre. El cliente 2 se queda en P. *Bajo fair choice vale.* En esa traza, pedido_2 está habilitada infinitas veces (en $(0, P, P)$ y en $(0, U, P)$), con lo cual se tiene que tomar. A partir de ahí el servidor solo puede responderle al cliente 2 (el cliente 1, como mucho, termina de usar y queda bloqueado en su pedido), y el cliente 2 pasa a U.

En Go el `select` elige al azar entre los casos listos, con lo cual en la práctica no elige siempre al mismo. Pero el LTS lo modela como no determinismo, y para concluir que el cliente 2 es atendido hace falta suponer fairness.

Versión B: vale, sin fairness. Cuando el servidor está en 2 (espera el pedido del cliente 2), el cliente 1 a lo sumo termina de usar y se bloquea en pedido_1. Lo único que puede pasar es que el cliente 2 termine de usar (si estaba en U) y mande su pedido, y después el servidor está obligado a responderle. Como el servidor pasa por 2 en cada vuelta, el cliente 2 usa una respuesta infinitas veces.

2. $\Box(\mathit{pide}_1 \to \Diamond \mathit{espera}_1)$.

Versión A: no vale, por lo mismo, al revés. El `select` elige siempre al cliente 2 y el cliente 1 se queda en P. *Bajo fair choice vale,* por el mismo argumento que la anterior.

Versión B: vale, sin fairness. Es el argumento simétrico: cuando el servidor está en 0, el único pedido que puede recibir es el del cliente 1.

3. $\Box\neg(\mathit{usa}_1 \land \mathit{usa}_2)$.

No vale en ninguna de las dos. En A, $(0, P, P) \xrightarrow{\text{pedido}_1} (1, E, P) \xrightarrow{\text{resp}_1} (0, U, P) \xrightarrow{\text{pedido}_2} (2, U, E) \xrightarrow{\text{resp}_2} (0, U, U)$. En B, la misma secuencia de acciones lleva a $(0, U, U)$. El servidor no espera a que un cliente termine de usar para atender al otro. *Fair choice no cambia nada.* Es una propiedad de safety, y la violación es un prefijo finito. Cualquier forma de continuar esa traza sigue violándola.

4. $\Box(\mathit{espera}_1 \to \bigcirc(\mathit{espera}_1 \lor \mathit{usa}_1))$.

Vale en las dos. El siguiente paso es del sistema, no del cliente 1, así que puede ser una acción del cliente 2 o del servidor con el cliente 2. En ese caso el cliente 1 sigue en E. Si el paso es del cliente 1, la única acción que tiene desde E es resp_1, que lo lleva a U. Como no hay estados sin salida, siempre hay un siguiente paso. El error típico es leer $\bigcirc$ como "el siguiente paso del cliente 1" y concluir que la fórmula dice que la respuesta llega enseguida.

Ejercicio 4.

Consideramos dos algoritmos *divide and conquer* secuenciales para calcular la función `maxSubArray` que, dado un arreglo de enteros `A` de longitud n (puede asumir n potencia de 2), determina el valor de la máxima suma de un subarreglo contíguo en A. Es decir, si por ejemplo tenemos el arreglo

$$[-2, 1, -3, 4, -1, 2, 1, -5, 4]$$

la suma máxima es 6, dada por el subarreglo $[4, -1, 2, 1]$. Se pide:

a) Adapte el pseudocódigo de estos algoritmos para que trabajen bajo un esquema *fork-join*.

b) Para ambos, calcule el *work*, el *span*, y el paralelismo.

c) En función de los resultados obtenidos, concluya cuál de las dos implementaciones es más conveniente (desde el punto de vista teórico) para su uso en una computadora de alta capacidad de paralelismo (explique el *speedup* alcanzable, por ejemplo, para una entrada de tamaño 2^{10} en una computadora con 64 procesadores, basándose en la aproximación $T_P \approx T_1/P + T_\infty$).

Resolvemos este problema de las siguientes maneras:

a) Variante (I):

```
int maxSubArray(int[] A, int lo, int hi) {
    if (hi - lo == 1) {
        return A[lo];
    }
    int mid = (lo + hi) / 2;
    int low = maxSubArray(A, lo, mid);
    int high = maxSubArray(A, mid, hi);
    int cruza = mejorSumaLineal(A, mid-1, lo-1)  // mejor sufijo de la parte baja
              + mejorSumaLineal(A, mid, hi-1);   // mejor prefijo de la parte alta
    return max(low, high, cruza);
}
```

Donde la respuesta es `maxSubArray(A, 0, len(A)-1)`. La función `mejorSumaLineal(A, desde, hasta)` recorre *secuencialmente*, en $\Theta(|desde - hasta|)$ pasos, las posiciones desde `desde` hacia `hasta`, y devuelve la máxima suma de un segmento que arranca en la posición `desde` y avanza hacia `hasta` (por lo que hacia la derecha da el mejor prefijo y hacia la izquierda da el mejor sufijo del subarreglo).

b) Variante (II):

```
Tupla mejoresCasos(int[] A, int lo, int hi) {
    // Tupla(total, optimo, prefijo, sufijo)
    if (hi - lo == 1) {
        return new Tupla(A[lo], A[lo], A[lo], A[lo]);
    }
    int mid = (lo + hi) / 2;
    Tupla b = mejoresCasos(A, lo, mid);
    Tupla d = mejoresCasos(A, mid, hi);
    int total   = b.total + d.total;
    int optimo  = max(b.optimo, d.optimo, b.sufijo + d.prefijo);
    int prefijo = max(b.prefijo, b.total + d.prefijo);
    int sufijo  = max(d.sufijo, d.total + b.sufijo);
    return new Tupla(total, optimo, prefijo, sufijo);
}
```

Donde la respuesta es `mejoresCasos(A, 0, len(A)-1).optimo`.

Solución.

a) En ambos casos replicamos la dinámica del `FibTask` de la teórica: se lanza la recursión sobre una mitad con `fork()`, la otra mitad se ejecuta en la propia tarea con `compute()`, y el resultado de la lanzada se obtiene con `join()` (tratando de demorarlo lo más posible). La parte de combinación queda igual. Notar que la adaptación no altera el cómputo.

Convertimos las funciones a tareas (`RecursiveTask`) para poder hacerles el `fork` y el `join` que necesitamos (no es necesario usar la sintaxis exacta).

Variante (I).

```

class MaxSubArrayTask extends RecursiveTask<Integer> {
    int [] A;
    int lo;
    int hi;

    public MaxSubArrayTask(int[] A, int lo, int hi){
        //setea los valores
    }

    protected Integer compute() {
```

```
        if (hi - lo == 1) {
            return A[lo];
        }
        int mid = (lo + hi) / 2;
        MaxSubArrayTask low = new MaxSubArrayTask(A, lo, mid);
        low.fork();
        int high = new MaxSubArrayTask(A, mid, hi).compute();
        int cruza = mejorSumaLineal(A, mid-1, lo-1)  // mejor sufijo de la parte baja
                  + mejorSumaLineal(A, mid, hi-1);   // mejor prefijo de la parte alta
        return max(low.join(), high, cruza);
    }

}
```

La solución se obtiene con `(new MaxSubArrayTask(A, 0, len(A)-1)).compute()`

Variante (II).

```
// Se asume Tupla(total, optimo, prefijo, sufijo)
class MejoresCasosTask extends RecursiveTask<Tupla> {
    int [] A;
    int lo;
    int hi;

    public MejoresCasosTask(int[] A, int lo, int hi) {
        //setea los valores
    }

    protected Tupla compute() {
        if (hi - lo == 1) {
            return new Tupla(A[lo], A[lo], A[lo], A[lo]);
        }
        int mid = (lo + hi) / 2;
        MejoresCasosTask low = new MejoresCasosTask(A, lo, mid);
        low.fork();
        Tupla r = new MejoresCasosTask(A, mid, hi).compute();
        Tupla l = low.join();
        int total   = l.total + r.total;
        int optimo  = max(l.optimo, r.optimo, l.sufijo + r.prefijo);
        int prefijo = max(l.prefijo, l.total + r.prefijo);
        int sufijo  = max(r.sufijo, l.sufijo + r.total);
        return new Tupla(total, optimo, prefijo, sufijo);
    }

}
```

b) **Work, span y paralelismo.**

Variante (I).

Work: Cada llamada parte el rango en dos mitades y las combina con dos recorridos lineales que juntas cubren el segmento completo. El Work esencialmente da lo mismo que hacer la ejecución secuencial:

$$T_1(n) = 2\,T_1(n/2) + \Theta(n), \qquad T_1(1) = \Theta(1).$$

Caso 2 del teorema maestro (Tanto CLRS3 como CLRS4) ($\log_2 2 = 1 = f(n)$):

$$T_1(n) = \Theta(n \log n).$$

Span: Las dos mitades corren en paralelo (aportan un máximo, no una suma), pero los recorridos lineales del combine son secuenciales y van enteros al camino crítico, igual que el `merge` secuencial del mergesort:

$$T_\infty(n) = T_\infty(n/2) + \Theta(n) = \Theta(n + n/2 + n/4 + \dots) = \Theta(n).$$

Esta derivación del *span* también sale con el caso 3 del teorema maestro; sólo hay que tener en cuenta que la condición de ese caso es $f(n) \in \Omega(n^{\log_b a + \varepsilon})$, y que hay que hacer el chequeo de regularidad: tiene que existir $c < 1$ tal que $af(n/b) \le cf(n)$. Para la recurrencia del *span*, $a = 1$, $b = 2$ (con lo cual $n^{\log_b a} = n^0 = 1$) y $f(n) = \Theta(n)$, que es $\Omega(n)$ con $\varepsilon = 1$; para la regularidad, $af(n/2) = n/2 \le cn$, que ciertamente vale para $c = \frac{1}{2}$.

Nota: Se podría pensar en crear una tarea para `mejorSumaLineal`, lo cual permitiría paralelizar el avance en ambas direcciones. Eso mejoraría el cómputo, pero sólo en una constante; para el análisis asintótico mantiene el *work* y el *span*. Sin embargo, es posible hacer esta búsqueda de prefijo/sufijo de manera paralela: el algoritmo que lo hace se llama *escaneo de Blelloch* (usándolo, el span bajaría a $\Theta(\log^2 n)$, pero el *work* seguiría siendo $\Theta(n \log n)$).

Paralelismo:

$$\frac{T_1(n)}{T_\infty(n)} = \frac{\Theta(n \log n)}{\Theta(n)} = \Theta(\log n).$$

Variante (II).

Work: Cada llamada parte en dos mitades y combina en $\Theta(1)$ (constantes comparaciones y sumas):

$$T_1(n) = 2\,T_1(n/2) + \Theta(1), \qquad T_1(1) = \Theta(1).$$

Caso 1 del teorema maestro:

$$T_1(n) = \Theta(n).$$

Span: Las dos mitades corren en paralelo y la combinación es $\Theta(1)$:

$$T_\infty(n) = T_\infty(n/2) + \Theta(1) = \Theta(\log n).$$

Es el caso 2 del teorema maestro (nuevamente, aplica para CLRS3 y CLRS4).

Paralelismo:

$$\frac{T_1(n)}{T_\infty(n)} = \Theta(n/\log n).$$

c) La variante (II) domina a la variante (I) en las tres métricas: menos pasos totales ($\Theta(n)$ contra $\Theta(n \log n)$), camino crítico más corto ($\Theta(\log n)$ contra $\Theta(n)$) y mucho más paralelismo ($\Theta(n/\log n)$ contra $\Theta(\log n)$).

Para el caso $n = 2^{10} = 1024$ (claramente $\log_2 n = 10$):

- *Variante (I): work* $\Theta(n \log n) = 1024 \cdot 10$ pasos, *span* $\Theta(n) = 1024$ pasos, paralelismo $\Theta(\log n) = 10$. Aunque hubiera infinitos procesadores, la cadena de recorridos lineales del camino crítico requiere al menos 1024 pasos. El *speedup* máximo teórico es de unas 10 veces, y encima el trabajo total es 10 veces el de la variante (II).
- *Variante (II): work* $\Theta(n) = 1024$ pasos, *span* $\Theta(\log n) = 10$ pasos, paralelismo $\Theta(n/\log n) \approx 102$: hay trabajo paralelo suficiente para aprovechar razonablemente hasta unos cientos de procesadores.

Usando la aproximación *greedy* $T_P \approx T_1/P + T_\infty$, con $P = 64 = 2^6$ procesadores (tomando las constantes en Θ igual a 1):

$$\text{(I): } T_{64} \approx \frac{1024 \cdot 10}{2^6} + 1024 = 160 + 1024 = 1184 \qquad \text{contra} \qquad \text{(II): } T_{64} \approx \frac{1024}{2^6} + 10 = 16 + 10 = 26.$$

Es decir, con 64 procesadores la variante (II) tarda 45 **veces menos pasos**. En términos del *speedup alcanzable* (respecto de la ejecución secuencial de cada variante, es decir, T_1/T_{64}): la variante (I) logra $10240/1184 \approx 8{,}6$, menos incluso que su paralelismo teórico de 10, mientras que la variante (II) logra $1024/26 \approx 39$, y puede seguir creciendo con P hasta el límite que marca su paralelismo (≈ 102). Por eso (II) es la indicada para una computadora de alta capacidad de paralelismo. Esencialmente en la variante (I) la combinación secuencial $\Theta(n)$ aplasta el paralelismo (un poco lo que pasa en el ejercicio de la práctica de mergesort con `merge` secuencial), mientras que en la variante (II) cada nivel del árbol recursivo se computa en paralelo y solo se paga $\Theta(\log n)$ de profundidad.